

Embedded C and CMSIS

Introduction to ARM Cortex-M Microprocessors – Chapter 4

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1** Why This Matters
- 2 C and the Compiler
- 3 CMSIS
- 4 Writing to Hardware

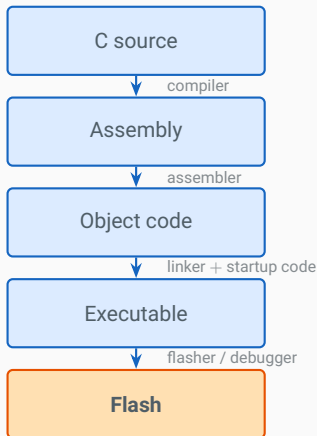
This chapter joins the two halves of the book. You know C. You know how the processor and memory are organized. Now we connect them: a C variable becomes a register, a C structure becomes a peripheral, and one assignment becomes a voltage on a pin. This is where hardware first moves.

After this chapter, you can:

1. Predict the assembly the compiler generates for simple C.
2. Explain the CMSIS components and which one does register-level work.
3. Show how a C structure overlays a peripheral's registers.
4. Justify `volatile` with two optimizations it prevents.
5. Apply the set, clear, toggle and test idioms.
6. Write a bare-metal CMSIS program that drives a GPIO pin.

- 1 Why This Matters
- 2 **C and the Compiler**
- 3 CMSIS
- 4 Writing to Hardware

What the Compiler Does



Each step is a separate program. You can inspect each one. This is how you find out what your C code actually became.

Why you learned assembly first

You will write C for the rest of this book. But when something goes wrong, the debugger shows you **instructions**, not C.

Chapters 5–7 exist so this disassembly window is readable, not frightening.

The linker adds **startup code**. It sets the stack pointer, zeroes your global variables, and calls `main()`, all before any line you wrote runs.

	C	Assembly
Development speed	Fast	Slow
Readability	Good	Poor
Portability	Across ARM families	None
Code size	Good, with a modern compiler	Smallest possible
Hardware access	Through pointers and CMSIS	Direct
Typical use	Almost everything	Startup, ISR entry, hand-tuned DSP

What C gives you, and what it does not

C gives you the same hardware access as assembly, with far less bookkeeping. What it does *not* give you is **protection**. There is no runtime check to catch a bad pointer.

1 Why This Matters

2 C and the Compiler

3 **CMSIS**

4 Writing to Hardware

Cortex Microcontroller Software Interface Standard. This is ARM's layer between your C code and the silicon. It works the same way across different chip vendors.

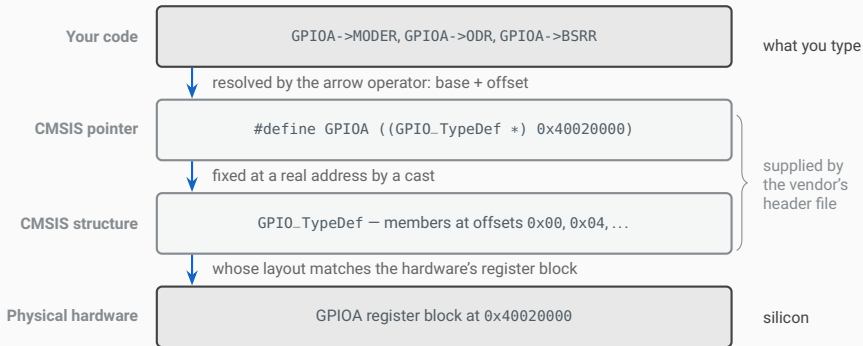
What it gives you

- A **struct** per peripheral, laid out on the real register offsets
- A **pointer** to each peripheral at its base address
- Named constants for bit fields
- Core functions: `__enable_irq()`, `NVIC_EnableIRQ()`, `__WFI()`

You need just one header: `#include "stm32f446xx.h"`

Why it matters

- The **same names** across every vendor's Cortex-M
- Readable: `GPIOA->ODR` rather than `*(volatile uint32_t*)0x40020014`
- The struct layout does the address arithmetic for you



You only need to think about the top and bottom layers: the hardware exists, and your code names its registers. The two middle layers are a struct and a cast. They are written once, in a header file that you never edit.

```
typedef struct {  
    __IO uint32_t MODER;  
    __IO uint32_t OTYPER;  
    __IO uint32_t OSPEEDR;  
    __IO uint32_t PUPDR;  
    __IO uint32_t IDR;  
    __IO uint32_t ODR;  
    __IO uint32_t BSRR;  
    __IO uint32_t LCKR;  
    __IO uint32_t AFR[2];  
} GPIO_TypeDef;
```

Each member is one **32-bit register**, in hardware order. A `uint32_t` is 4 bytes, so member n sits at offset $4n$. These are exactly the offsets given in the reference manual.

`__IO` expands to `volatile`. CMSIS marks every register for you.

`AFR[2]` uses Chapter 3's array indexing: two real registers, reached as `AFR[0]` and `AFR[1]`.

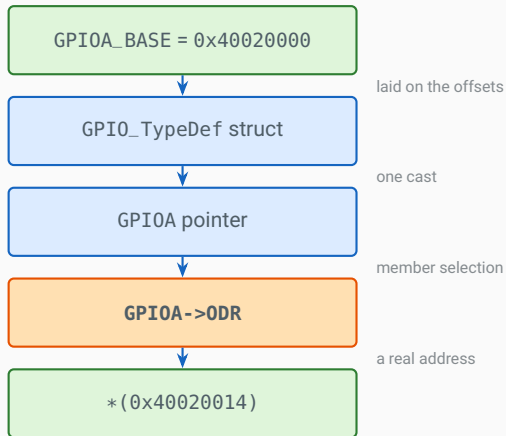
From Name to Address

```
#define GPIOA_BASE 0x40020000UL
```

```
#define GPIOA \
((GPIO_TypeDef *) GPIOA_BASE)
```

A cast turns a plain number into a pointer to the struct. After this, the compiler computes every register's address for you.

ODR is the sixth member, so its offset is $5 \times 4 = 20 = 0x14$. This gives **0x40020014**, the same address as in Chapter 1.



1 Why This Matters

2 C and the Compiler

3 CMSIS

4 Writing to Hardware

volatile Two optimisations it prevents

1. Caching a value in a register

```
while ((*status & 1) == 0)
;    // waits forever without
    // volatile
```

Without `volatile`, the compiler reads the address once, and never checks it again.

2. Discarding a “redundant” write

```
*reg = 1;
*reg = 0;    // without volatile the
             // first write is deleted
```

Both writes matter if the address is a **pulse output**.

The rule

Every pointer to a hardware register must be `volatile`. So must every variable shared with an interrupt handler.

A compiler may assume that memory only changes when the program changes it. For a status register updated by hardware, that assumption is wrong.

Building a Mask: The Shift

Every bit idiom starts with $1U \ll n$. Begin with $1U$: only bit 0 is set. Shift it left by n , and the **only** set bit is now bit n . This value is called a **mask**.

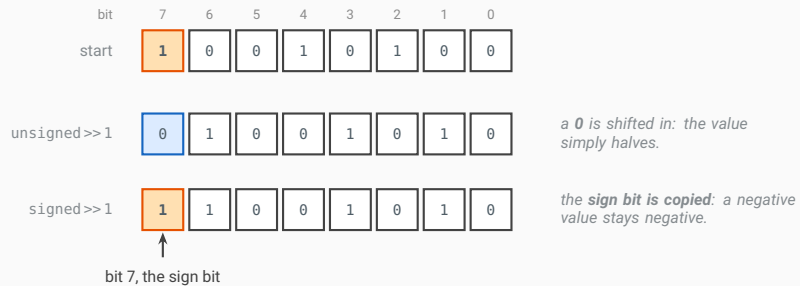


The shift count *is* the bit number

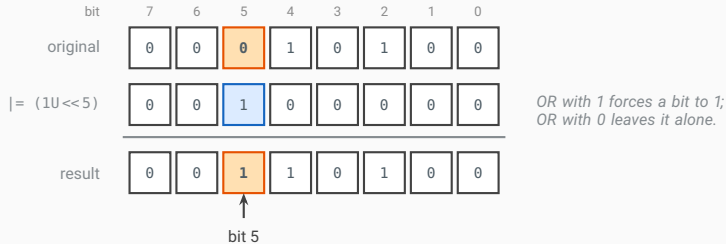
$1U \ll 5$ selects bit 5, not bit 6. Writing the pin number directly in the shift keeps the code readable, and easy to change for a different pin.

Why the U Suffix

A plain 1 is signed. Its top bit is the sign bit. C treats signed and unsigned shifts differently, and this difference matters exactly here.

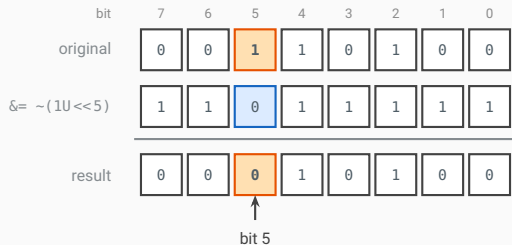


1 << 31 is **undefined behaviour** on a 32-bit int. 1U << 31 is fully defined, and gives 0x80000000. A register is 32 switches, not a signed number. Always write U.



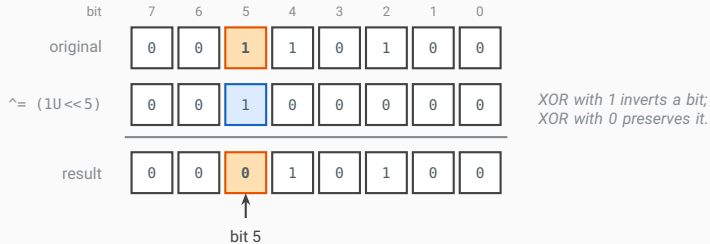
`GPIOA->ODR |= (1U << 5);` — only the masked bit changes; every other bit is OR-ed with 0 and passes through untouched.

Clearing a Bit: $\&=$ with an Inverted Mask



AND with 0 forces a bit to 0;
AND with 1 leaves it alone.

`GPIOA->ODR &= ~(1U << 5);` – the tilde makes every bit except bit 5 a 1, so only bit 5 is forced low. The compiler emits BIC.



`GPIOA->ODR ^= (1U << 5);` – the same statement run twice returns the register to its starting value, which is why one line suffices to blink an LED.

Multi-Bit Fields: Clear, Then Set

OR alone cannot clear a bit. Say MODER bits 11:10 read 11, and you OR in 01. The result is still 11. You must clear the field first, before setting it.

bit	15	14	13	12	11	10	9	8
original	0	0	0	0	1	1	0	0
$\&= \sim(3U << 10)$	1	1	1	1	0	0	1	1
cleared	0	0	0	0	0	0	0	0
$ = (1U << 10)$	0	0	0	0	0	1	0	0
result	0	0	0	0	0	1	0	0

bits 11:10

OR alone cannot clear a bit, so a multi-bit field must be zeroed first...

...and only then written with the value you want.

Goal	In C	Compiles to
Set bit n	<code>REG = (1U << n);</code>	ORR
Clear bit n	<code>REG &= ~(1U << n);</code>	BIC
Toggle bit n	<code>REG ^= (1U << n);</code>	EOR
Test bit n	<code>if (REG & (1U << n))</code>	TST

Read-modify-write

A peripheral register holds settings for **many** pins. You must keep the bits you are not changing. `|=` and `&=` do this automatically. A plain `=` would wipe out every other setting.

A multi-bit field needs a **mask**: first clear it with `&= ~(3U << pos)`, then set it with `|= (value << pos)`. The mask is always “all ones for this field.”

A Complete Bare-Metal Program

```
#include "stm32f446xx.h"

int main(void)
{
    // 1. Enable the clock for GPIOA
    RCC->AHB1ENR |= (1U << 0);

    // 2. Set PA5 as an output
    GPIOA->MODER &= ~(3U << (5*2));
    GPIOA->MODER |= (1U << (5*2));

    // 3. Toggle PA5 forever
    while (1) {
        GPIOA->ODR ^= (1U << 5);
        for (volatile int i = 0;
            i < 1000000; i++);
    }
}
```

Every idea in this chapter

- Struct-based register access
- Bit masking and shifting
- Read-modify-write
- Clock enable, then configure

Why 5*2? MODER gives each pin **two** bits, so pin 5's bits start at position 10.

Why volatile on i? Without it, the optimiser deletes the whole delay loop.

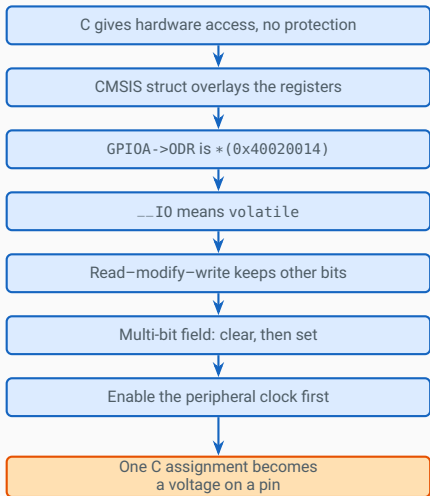
```
RCC->AHB1ENR |= (1U << 0);
```

- Every peripheral starts with its **clock disabled**, to save power.
- A peripheral with no clock does not respond to writes. They are simply **lost**.
- RCC is the Reset and Clock Control peripheral. GPIOA is bit 0 of AHB1ENR.

The classic first bug

You write the GPIO configuration. The code compiles and runs. But **nothing happens**. No fault, no error: the writes went nowhere, because the clock was off.

Enable the clock first. Always.



Terminology covered

- CMSIS, device header
- Struct overlay
- volatile, __IO
- Set, clear, toggle, test
- Peripheral clock enable

Next: Chapter 5

ARM Assembly Fundamentals

We read what this C compiles to.